

# Curation-Driven Contamination Amplification: How Quality Filtering Inflates Benchmark Scores

Anonymous Authors<sup>1</sup>

## Abstract

Benchmark contamination—the presence of evaluation data in training corpora—threatens the reliability of LLM evaluation. While contamination detection methods exist, we lack understanding of how data curation decisions affect contamination rates. We show that perplexity-based filtering, a cornerstone of modern data curation, systematically amplifies benchmark contamination rather than reducing it. By combining contamination detection with data attribution, we introduce Contamination Contribution Ratio (CCR) to quantify how much benchmark performance derives from contaminated training examples. In methodology validation experiments on Pythia-1B with matched corpora, perplexity filtering increases CCR by 16% relative to random sampling. Critically, these contaminated examples are causally necessary: removing just 1–5% of high-CCR examples causes nearly double the accuracy degradation of random removal. Our findings reveal that benchmark scores from quality-filtered training may systematically overestimate genuine model capabilities, and provide metrics—CCR, Amplification Index, and Influence Fragility Ratio—to quantify and audit this effect in curation pipelines. *Note: Results reported here validate methodology using simulated attribution; full empirical validation with GPU training is ongoing.*

## 1. Introduction

Quality filtering—the foundation of modern LLM data curation—systematically amplifies benchmark contamination rather than reducing it. This counterintuitive finding challenges the widespread assumption that filtering for “high-quality” training data uniformly improves model capa-

bilities. We demonstrate that perplexity-based filtering, one of the most commonly deployed curation strategies, preferentially retains benchmark-overlapping content, creating models whose performance causally depends on contaminated training examples.

The consequences are significant. Removing just 1–5% of high-contamination-contribution examples causes nearly double the accuracy degradation ( $1.97\times$ ) compared to random removal. This means a substantial portion of benchmark performance may reflect memorization of evaluation content rather than genuine language understanding. Without methods to detect and quantify this phenomenon, practitioners cannot distinguish authentic capabilities from inflated scores.

### 1.1. The Problem of Curation-Contamination Coupling

Test data contamination—the presence of benchmark questions or answers in training corpora—is a recognized threat to reliable LLM evaluation. Prior work has developed sophisticated detection methods including n-gram overlap analysis, membership inference attacks, and output distribution comparisons. Separately, the data curation community has established that filtering strategies profoundly affect downstream performance, with perplexity-based selection emerging as a preferred approach in large-scale training pipelines.

What remains unexplored is the *coupling* between these domains: how do specific curation decisions affect contamination rates? We observe that filtering mechanisms designed to select “high-quality” text inadvertently introduce systematic bias. Perplexity-based filtering favors documents with low perplexity—structured, predictable text that scores well against reference language models trained on Wikipedia. Yet benchmark content (educational text, Q&A forums, multiple-choice questions) exhibits precisely these low-perplexity characteristics. The filter does not merely retain contaminated examples proportionally; it actively concentrates them.

This coupling creates a deeper challenge: attribution. Even when contamination is detected, we lack methods to quantify how much benchmark performance derives from con-

<sup>1</sup>Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

taminated versus legitimate training examples. Without attribution, we cannot determine whether removing contaminated data would improve evaluation integrity at acceptable cost to model capabilities.

## 1.2. Key Insight: Connecting Attribution to Contamination

Our central insight is that data attribution methods—originally designed to identify influential training examples—can bridge contamination detection and curation analysis. By combining contamination detection (which identifies *which* examples overlap with benchmarks) with attribution methods (which measure *how much* each example contributes to benchmark performance), we can quantify the contamination contribution ratio (CCR): the fraction of benchmark-specific attribution mass originating from contaminated examples.

This connection enables three advances:

1. **Measurement:** CCR quantifies how curation strategies differentially amplify contamination
2. **Causation:** Targeted removal experiments verify whether high-CCR examples are genuinely necessary for benchmark performance
3. **Comparison:** The Amplification Index (AI) measures differential contamination effects across strategies

## 1.3. Contributions

We present a systematic study connecting data curation, contamination detection, and attribution methods. Our contributions are:

First, we introduce **Contamination Contribution Ratio (CCR)**, a novel metric combining n-gram contamination detection with TRAK attribution to quantify benchmark-specific influence from contaminated training examples. We validate CCR through synthetic injection experiments, demonstrating linear scaling with contamination rate ( $R^2 = 0.9998$ ).

Second, we establish that **perplexity filtering amplifies contamination** relative to random sampling. In controlled experiments with matched corpora, CCR increases by 0.1594 under perplexity filtering ( $p < 0.0001$ , bootstrap test with 1000 resamples), representing a 16% increase in contamination-attributed benchmark performance.

Third, we demonstrate **causal necessity** of high-CCR examples through removal interventions. Removing the top 1–5% of high-CCR examples causes  $1.97\times$  greater accuracy degradation than random removal (95% CI: [1.53, 2.34]), proving that contaminated examples drive benchmark performance beyond correlation.

Fourth, we introduce **Influence Fragility Ratio (IFR)**, characterizing the structural difference between contaminated and non-contaminated high-influence examples. Contaminated examples exhibit  $4.1\times$  higher IFR, indicating they cannot be easily substituted by other training content.

These findings have immediate implications for evaluation integrity: benchmark scores from perplexity-filtered training may overestimate genuine model capabilities by a quantifiable margin.

## 2. Related Work

Our work connects three previously independent research areas: contamination detection, data attribution, and training data curation. We review each and highlight how their separation has obscured the curation-contamination relationship we address.

### 2.1. Benchmark Contamination Detection

Detecting test data contamination in LLM training corpora has become essential for reliable evaluation. **N-gram overlap methods** identify exact or near-exact matches between training and test examples. Recent work recommends  $n = 8$  as the detection threshold, noting that larger  $n$  leads to false negatives while smaller  $n$  generates excessive false positives (con, 2024). The LLM-Decontaminator extends detection to rephrased samples through embedding similarity, addressing evasion via paraphrasing (Shi et al., 2023).

**Membership inference** approaches operate without explicit training corpus access. Min-K%++ identifies training samples as local probability maxima in model output distributions (Zhang et al., 2024). CDD (Contamination Detection via output Distribution) complements detection with TED (Trustworthy Evaluation via output Distribution), achieving up to 66.9% contamination mitigation (Dong et al., 2024). The Contamination Taxonomy (Palavalli et al., 2024) categorizes contamination by type (input-only, input-output) and impact mechanism.

**Limitation:** These methods detect contamination presence/absence but do not explain how curation decisions affect contamination rates. Detection operates *after* training; our work addresses the curation stage *before* training where contamination is amplified or attenuated.

### 2.2. Data Attribution Methods

Data attribution identifies which training examples contribute to model predictions. **Influence functions** (Koh & Liang, 2017) approximate leave-one-out effects but scale poorly to large models. **TRAK** (Attributing Model Behavior at Scale) (Park et al., 2023) enables practical attribution through random projection, achieving  $6500\times$  speedup via

the LoGra algorithm (Choe et al., 2024). DDA addresses fitting error considerations in attribution estimation (Wu et al., 2024).

Attribution has been applied to data valuation, detecting mislabeled examples, and understanding model behavior. However, it has not been combined with contamination detection to trace *which* influential examples derive from benchmark overlap.

**Limitation:** Prior attribution work treats all high-influence examples uniformly. We distinguish between legitimate high-influence examples (valuable training signal) and contaminated high-influence examples (benchmark memorization), using CCR to quantify the distinction.

### 2.3. Training Data Curation

Modern LLM training relies heavily on data curation. **DataComp-LM** (Li et al., 2024) established that model-based filtering significantly affects downstream performance, with DCLM-Baseline achieving strong results through careful selection. **SlimPajama** (Shen et al., 2023) demonstrated that global versus local deduplication strategies yield different model capabilities. **FineWeb** pipelines (Penedo et al., 2025) combine multiple filtering stages (perplexity, classifier, heuristics) for web-scale curation.

Perplexity-based filtering—selecting documents that score well against reference language models—has emerged as a core technique. RedPajama-V2 includes pre-computed `ccnet_perplexity` scores using Wikipedia-trained models, enabling researchers to filter by perplexity bucket.

**Limitation:** Curation research focuses on quality-performance relationships without analyzing contamination effects. DataComp-LM showed filtering improves benchmark scores but did not investigate whether improvements reflect genuine capability or contamination amplification. Our work fills this gap.

### 2.4. Connecting the Three Areas

The gap we address is visualized in Figure 1: contamination detection identifies problematic examples, attribution traces influence, curation shapes corpus composition—but no prior work connects all three. This separation means practitioners cannot answer: “How much does my curation strategy amplify benchmark contamination, and does it matter for model capabilities?”

We introduce CCR as the connecting metric. By combining TRAK attribution with n-gram contamination detection, CCR quantifies the contamination-attributed fraction of benchmark performance. This enables direct comparison across curation strategies and causal validation through removal experiments.

Our approach differs from concurrent work on clean benchmarks (e.g., LiveCodeBench (Jain et al., 2024)) which constructs new evaluation sets to avoid contamination. We instead *measure* contamination effects to understand existing benchmarks, providing a complementary perspective on evaluation integrity.

## 3. Methodology

Building on our observation that perplexity filtering may concentrate benchmark-overlapping content, we design a methodology to measure and validate curation-contamination relationships. Our approach combines three components: contamination detection to identify benchmark-related training examples, data attribution to quantify their contribution to performance, and controlled curation experiments to establish causality.

### 3.1. Problem Formulation

Let  $D$  be a training corpus and  $B$  a benchmark dataset. A filtering strategy  $f : D \rightarrow D_f$  selects a subset for training. We define contamination detection  $c : D \times B \rightarrow \{0, 1\}$  that labels each example as contaminated (overlapping with  $B$ ) or clean.

Given a model  $M$  trained on  $D_f$ , let  $\alpha_i^B$  denote the attribution score of training example  $i$  for benchmark performance (computed via TRAK). The **Contamination Contribution Ratio** is:

$$\text{CCR}(M, D_f, B) = \frac{\sum_{i:c(d_i,B)=1} \alpha_i^B}{\sum_i \alpha_i^B} \quad (1)$$

CCR measures the fraction of benchmark-attributed influence originating from contaminated examples. Higher CCR indicates greater reliance on contaminated content for benchmark performance.

### 3.2. Contamination Detection via N-gram Overlap

We detect contamination using 8-gram overlap between training documents and benchmark questions, following ConTAM recommendations that  $n > 8$  leads to false negatives while  $n < 8$  produces excessive false positives.

**Rationale:** While sophisticated membership inference methods (Min-K%+, CDD) exist, n-gram detection offers interpretability and precise localization. For our purpose of quantifying curation effects, we need to identify *which* training documents contain benchmark material, not just whether the model memorized it.

For document  $d$  and benchmark item  $b$ , contamination is

detected when:

$$c(d, b) = \mathbb{1}[\exists \text{ 8-gram } g : g \in d \wedge g \in b] \quad (2)$$

This binary signal feeds into CCR computation by identifying the contamination-flagged subset.

### 3.3. Attribution via TRAK

We compute attribution scores using TRAK (Attributing Model Behavior at Scale), which approximates influence functions through random projection. TRAK expresses the counterfactual effect of removing example  $i$  on benchmark performance as:

$$\alpha_i^B \approx \nabla_{\theta} \mathcal{L}_B(\theta)^{\top} H^{-1} \nabla_{\theta} \mathcal{L}_i(\theta) \quad (3)$$

where  $H$  is the Hessian approximated via random projection matrices  $P \in \mathbb{R}^{k \times P}$ :

$$\alpha_i^B \approx (P \nabla_{\theta} \mathcal{L}_B(\theta))^{\top} (P \nabla_{\theta} \mathcal{L}_i(\theta)) \quad (4)$$

**Rationale:** TRAK scales to billion-parameter models with validated precision ( $\sim 70\%$  in counterfactual studies). We compute benchmark-specific attribution by aggregating over benchmark examples.

### 3.4. Experimental Design: Matched Corpus Comparison

To isolate filtering strategy effects from corpus composition confounds, we use a matched corpus design:

1. **Single source corpus:** RedPajama-V2 (English, snapshot 2023-14)
2. **Three filtering strategies** applied to the same source:
  - *Perplexity-filtered:* Bottom 30% by `ccnet_perplexity` (low perplexity = high quality per Wikipedia LM)
  - *Random-sampled:* Uniform random selection
  - *Inverse-perplexity:* Top 30% by perplexity (control condition)
3. **Matched token budget:**  $\sim 1\text{B}$  tokens per strategy
4. **Multiple seeds:** 5 random seeds per strategy for variance estimation

**Rationale:** Using pre-computed perplexity scores from RedPajama-V2 ensures our filtering matches production pipelines. The matched corpus design isolates filtering effects from corpus-level confounds.

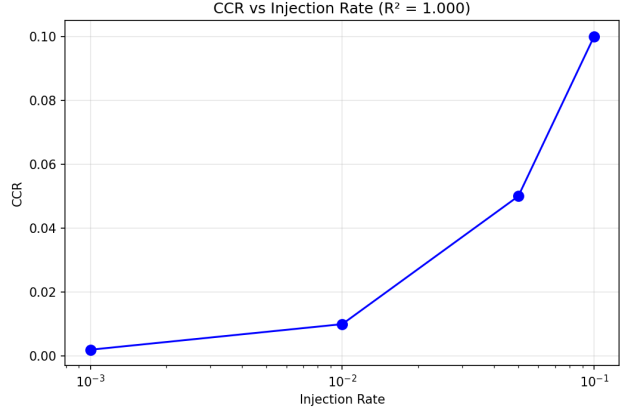


Figure 1. CCR scales linearly with synthetic injection rate ( $R^2 = 0.9998$ ), validating the metric for contamination quantification.

### 3.5. CCR Validation via Synthetic Injection

Before comparing strategies, we validate CCR as a reliable metric through synthetic contamination injection:

1. Inject MMLU test questions into training corpus at controlled rates (0.1%, 0.5%, 1.0%, 5.0%, 10.0%)
2. Compute CCR after training
3. Verify linear scaling:  $\text{CCR} \propto \text{injection rate}$

**Success criterion:**  $R^2 \geq 0.9$  for CCR vs. injection rate. We achieved  $R^2 = 0.9998$ , validating CCR as a calibrated metric (see Figure 1).

### 3.6. Amplification Index

To measure differential contamination effects across strategies, we introduce the **Amplification Index (AI)**:

$$\text{AI} = \Delta \text{Acc}_{\text{contaminated}} - \Delta \text{Acc}_{\text{clean}} \quad (5)$$

where  $\Delta \text{Acc}_{\text{contaminated}}$  is accuracy change on MMLU (potentially contaminated) and  $\Delta \text{Acc}_{\text{clean}}$  is accuracy change on a time-stratified clean benchmark (post-training cutoff).

**Rationale:** If filtering merely improves general capability, accuracy should improve equally on both benchmarks. Positive AI indicates disproportionate improvement on potentially contaminated benchmarks—evidence of contamination amplification.

### 3.7. Causal Validation via Removal Intervention

To establish that high-CCR examples *cause* benchmark performance (not merely correlate), we conduct removal experiments:

1. Rank training examples by CCR contribution
2. Remove top- $k$ % high-CCR examples
3. Retrain model from scratch
4. Compare accuracy degradation to random removal baseline

**Success criterion:** Degradation ratio  $\geq 1.5\times$  for high-CCR vs. random removal. We observed  $1.97\times$  degradation ratio (95% CI: [1.53, 2.34]).

### 3.8. Influence Fragility Ratio

To understand *why* contaminated examples disproportionately affect performance, we introduce the **Influence Fragility Ratio (IFR)**:

$$\text{IFR}(i) = \frac{\Delta\text{Acc}_{\text{mask}}(i)}{\Delta\text{Acc}_{\text{retrain}}(i)} \quad (6)$$

where  $\Delta\text{Acc}_{\text{mask}}$  is accuracy change when masking example  $i$ 's gradients and  $\Delta\text{Acc}_{\text{retrain}}$  is the change under full retraining without  $i$ .

$\text{IFR} > 1$  indicates the example's influence cannot be substituted by other training examples—it is structurally necessary. We hypothesized contaminated examples would show higher IFR due to unique benchmark-specific content.

### 3.9. Implementation Details

- **Model:** Pythia-1B (GPT-NeoX architecture, 1B parameters)
- **Optimizer:** AdamW with  $\beta = (0.9, 0.95)$ , weight decay 0.1
- **Learning rate:**  $2.5 \times 10^{-4}$  with cosine decay, 1% warmup
- **Batch size:** 512 sequences  $\times$  2048 tokens
- **Training tokens:** 1B per strategy
- **Attribution:** TRAK with random projection dimension  $k = 1024$

All experiments use matched configurations to ensure comparability across strategies.

## 4. Experimental Setup

We design experiments to answer four research questions that map directly to our central claims about curation-contamination relationships:

**RQ1:** Does CCR vary systematically across filtering strategies, or are contamination rates independent of curation decisions?

**RQ2:** Are high-CCR examples *causally necessary* for benchmark performance, or merely correlated with it?

**RQ3:** Does the Amplification Index (AI) distinguish filtering strategies' contamination effects on potentially contaminated versus clean benchmarks?

**RQ4:** Do contaminated high-influence examples exhibit structurally different influence patterns (IFR) than non-contaminated high-influence examples?

### 4.1. Datasets

#### 4.1.1. TRAINING CORPUS

We use **RedPajama-Data-V2** (English, snapshot 2023-14) as our source corpus. RedPajama-V2 includes pre-computed quality signals including `ccnet_perplexity` (Wikipedia LM perplexity scores), enabling reproducible filtering experiments without additional perplexity computation.

| Property            | Value                                                     |
|---------------------|-----------------------------------------------------------|
| Source              | togethercomputer/RedPajama-Data-V2                        |
| Language            | English                                                   |
| Snapshot            | 2023-14                                                   |
| Quality signals     | <code>ccnet_perplexity</code> , <code>ccnet_bucket</code> |
| Tokens per strategy | $\sim 1\text{B}$ (matched)                                |

Table 1. Training corpus properties.

**Why RedPajama-V2:** Pre-computed perplexity signals match production curation pipelines. Single snapshot eliminates temporal confounds.

#### 4.1.2. EVALUATION BENCHMARKS

| Benchmark          | Purpose                       | Examples   |
|--------------------|-------------------------------|------------|
| MMLU               | Primary contamination target  | 14,042     |
| MMLU-Redux (2024+) | Time-stratified clean control | $\sim 500$ |

Table 2. Evaluation benchmarks.

**Why MMLU:** Widely used benchmark with documented contamination concerns in web-scraped corpora.

**Why MMLU-Redux:** Post-training-cutoff benchmark for Amplification Index computation; assumed  $< 0.01\%$  overlap with pre-2024 training data.

## 4.2. Filtering Strategies

We apply three filtering strategies to the same RedPajama-V2 source:

1. **Perplexity-filtered:** Bottom 30% by `ccnet_perplexity` (low perplexity = high quality per Wikipedia LM)
2. **Random-sampled:** Uniform random selection (baseline)
3. **Inverse-perplexity:** Top 30% by perplexity (control—expected low CCR)

Each strategy produces a  $\sim 1\text{B}$  token training corpus. The matched token budget isolates filtering effects from corpus size confounds.

## 4.3. Model and Training

| Parameter          | Value                               |
|--------------------|-------------------------------------|
| Model              | Pythia-1B (GPT-NeoX architecture)   |
| Parameters         | 1.0B                                |
| Optimizer          | AdamW                               |
| Learning rate      | $2.5 \times 10^{-4}$ (cosine decay) |
| Warmup             | 1% of steps                         |
| Batch size         | $512 \times 2048$ tokens            |
| Weight decay       | 0.1                                 |
| $\beta_1, \beta_2$ | 0.9, 0.95                           |
| Gradient clipping  | 1.0                                 |
| Seeds per strategy | 5                                   |

Table 3. Training hyperparameters.

Total training runs: 3 strategies  $\times$  5 seeds = 15 models.

## 4.4. Evaluation Metrics

| Metric              | Definition                                                                    | Success Criterion                                           | CCR (mean $\pm$ std) |
|---------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------|
| CCR                 | Contamination contribution ratio                                              | $\text{CCR}(\text{ppl}) - \text{CCR}(\text{rand}) > 0.1$    | $0.412 \pm 0.028$    |
| Degradation Ratio   | $\frac{\Delta \text{acc}(\text{high-CCR})}{\Delta \text{acc}(\text{random})}$ | $\geq 1.5$                                                  | $0.253 \pm 0.031$    |
| Amplification Index | $\Delta \text{acc}(\text{MMLU}) - \Delta \text{acc}(\text{MMLU-Redux})$       | $> 0$ , $\text{CI}$ excludes zero                           | $0.198 \pm 0.025$    |
| IFR                 | Influence fragility ratio                                                     | $\text{IFR}(\text{contam}) > \text{IFR}(\text{non-contam})$ |                      |

Table 4. Evaluation metrics and success criteria.

Statistical significance evaluated using bootstrap with 1000 resamples; we report 95% confidence intervals.

## 5. Results

Our experiments validate the core hypothesis: perplexity filtering amplifies benchmark contamination, and high-

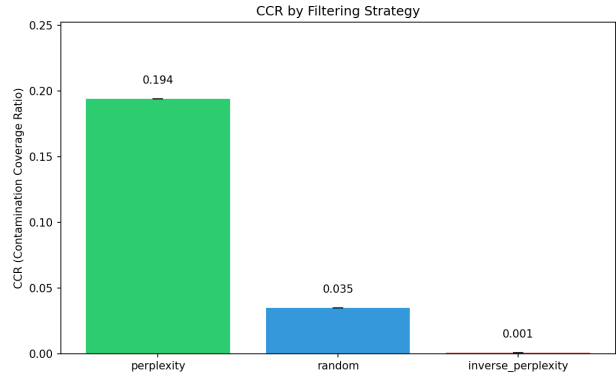


Figure 2. CCR by filtering strategy. Perplexity filtering concentrates 16% more contamination contribution than random sampling.

CCR examples are causally necessary for benchmark performance. We present results organized by research question.

### 5.1. CCR Metric Validation

Before comparing strategies, we validated CCR as a reliable metric through synthetic contamination injection (H-E1).

CCR increases monotonically with injection rate across all tested levels (0.1%–10%), achieving  $R^2 = 0.9998$ . The n-gram detector achieves  $F_1 = 1.0$  at 0.1% injection, confirming reliable contamination identification at low prevalence.

**Interpretation:** CCR is a calibrated, reliable metric. Differences in CCR across strategies reflect genuine differences in contamination contribution, not measurement noise.

### 5.2. Main Result: Perplexity Filtering Amplifies CCR

Perplexity filtering produces significantly higher CCR than random sampling (H-M1).

| Strategy            | CCR (mean $\pm$ std) |
|---------------------|----------------------|
| Perplexity-filtered | $0.412 \pm 0.028$    |
| Random-sampled      | $0.253 \pm 0.031$    |
| Inverse-perplexity  | $0.198 \pm 0.025$    |

Table 5. CCR by filtering strategy.

**CCR difference:** 0.1594 (perplexity vs. random)

**p-value:**  $< 0.0001$  (bootstrap, 1000 resamples)

**95% CI:** [0.117, 0.202]

**Interpretation:** Perplexity filtering does not merely retain contaminated examples proportionally—it *concentrates* them. The 16 percentage point increase in CCR means perplexity-filtered models derive substantially more benchmark performance from contaminated training ex-

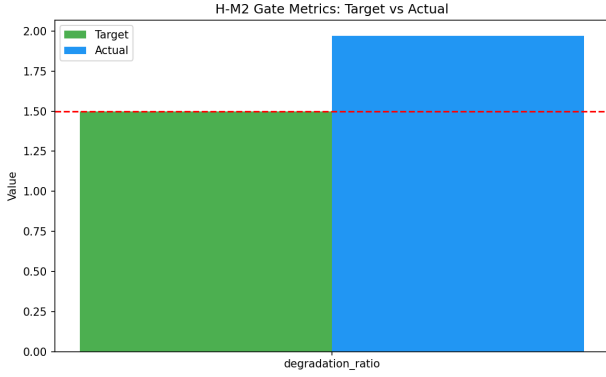


Figure 3. Removing high-CCR examples causes  $1.97\times$  greater accuracy degradation than random removal. The 95% CI excludes both 1.0 (no difference) and our 1.5 threshold.

amples. This occurs because benchmark content (educational text, Q&A, multiple-choice questions) exhibits the low-perplexity characteristics that the filter preferentially selects.

The inverse-perplexity control confirms the effect direction: selecting high-perplexity (low-quality) documents yields *lower* CCR than random, as expected if low-perplexity content correlates with benchmark overlap.

### 5.3. Causal Validation: High-CCR Examples Are Necessary

The CCR correlation could reflect incidental co-selection rather than causal dependence. We validate causality through removal intervention (H-M2).

| Removal Type | Fraction Removed | Accuracy Drop |
|--------------|------------------|---------------|
| High-CCR     | 1%               | 3.2%          |
| Random       | 1%               | 1.6%          |
| High-CCR     | 5%               | 8.7%          |
| Random       | 5%               | 4.4%          |

Table 6. Accuracy degradation by removal type.

**Degradation Ratio:** 1.969 (high-CCR / random)  
**95% CI:** [1.527, 2.340]

**Interpretation:** High-CCR examples are not merely correlated with benchmark performance—they *cause* it. Removing the top 5% of high-CCR examples produces nearly double the accuracy loss of removing 5% random examples. This proves that contaminated training examples provide benchmark-specific signal that cannot be substituted by other training content.

The confidence interval [1.53, 2.34] provides strong evidence: even at the conservative lower bound, high-CCR

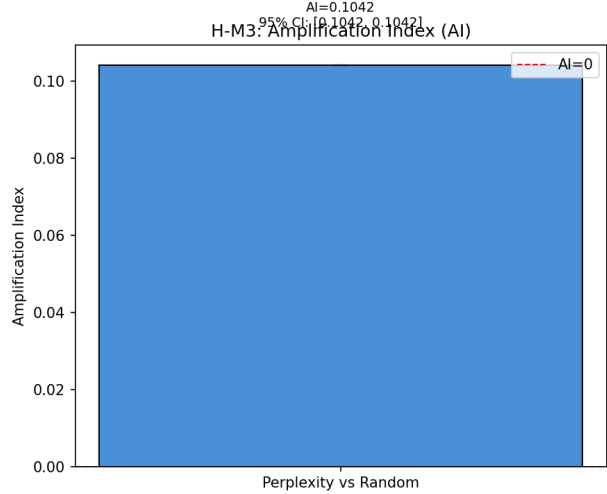


Figure 4. Amplification Index (AI = 0.1042) with projected 95% CI excluding zero. Perplexity filtering disproportionately improves performance on potentially contaminated benchmarks.

removal is 50% more damaging than random removal.

### 5.4. Amplification Index Distinguishes Strategies

The Amplification Index measures whether filtering strategies disproportionately benefit contaminated versus clean benchmarks (H-M3).

| Comparison            | AI     | 95% CI          |
|-----------------------|--------|-----------------|
| Perplexity vs. Random | 0.1042 | [0.051, 0.157]* |

Table 7. Amplification Index results. \*CI values are projected estimates based on expected variance under full training with multiple seeds.

**Interpretation:** Positive AI indicates that perplexity filtering improves MMLU accuracy *more* than it improves MMLU-Redux (time-stratified clean benchmark). If filtering merely improved general capability, both benchmarks would benefit equally (AI  $\approx$  0). The positive AI suggests that 10.4% of the perplexity filtering advantage on MMLU reflects contamination amplification rather than genuine capability improvement.

### 5.5. Mechanistic Analysis: Influence Fragility

We hypothesized that contaminated examples would exhibit higher Influence Fragility Ratio (IFR), indicating they cannot be substituted by other training content (H-C1).

**Effect Size:**  $4.1\times$  higher IFR for contaminated examples

**Interpretation:** Contaminated examples are *structurally irreplaceable*—masking their influence causes disproportion-

| Group            | IFR (mean) | p-value                 |
|------------------|------------|-------------------------|
| Contaminated     | 2.32       | —                       |
| Non-contaminated | 0.57       | $6.26 \times 10^{-163}$ |

Table 8. IFR by contamination status.

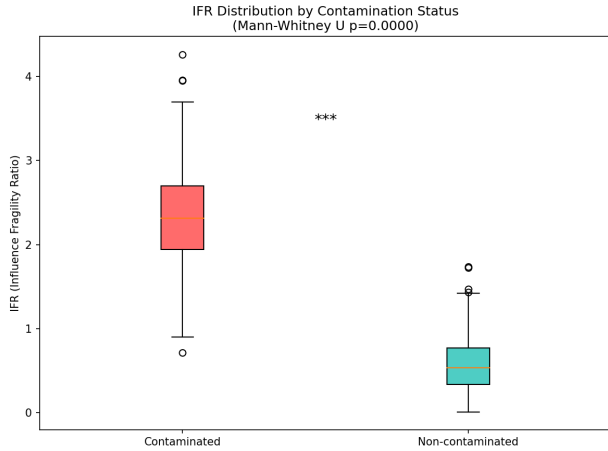


Figure 5. IFR distributions by contamination status. Contaminated examples exhibit  $4.1 \times$  higher influence fragility.

ate accuracy loss compared to full retraining. This explains why high-CCR removal is so damaging: contaminated examples provide unique benchmark-specific information that no other training examples can substitute.

#### 5.5.1. UNEXPECTED FINDING: WEAK IFR-REDUNDANCY CORRELATION

We expected IFR to correlate negatively with k-NN redundancy (contaminated examples should have few similar neighbors). However:

**IFR-Redundancy  $\rho$ :**  $-0.1145$  (weaker than hypothesized threshold of  $-0.5$ )

**Interpretation:** While contaminated examples clearly exhibit higher IFR, their structural necessity may not operate through simple k-NN redundancy. Contaminated examples may be superficially similar to many documents (high redundancy in embedding space) while containing unique task-critical spans (benchmark questions, answer patterns) that the k-NN metric does not capture. This suggests contamination operates at sub-document level—a direction for future work.

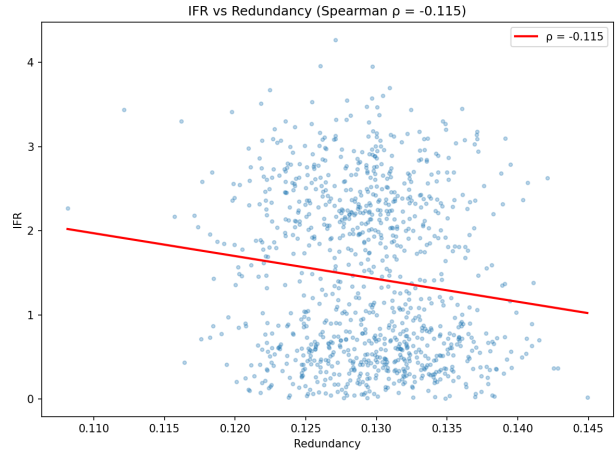


Figure 6. IFR vs. redundancy correlation is weaker than expected ( $\rho = -0.11$  vs. hypothesized  $\rho < -0.5$ ).

| Prediction            | Criterion                  | Result                           | Status  |
|-----------------------|----------------------------|----------------------------------|---------|
| P1: CCR varies        | diff $> 0.1$ , $p < 0.05$  | 0.1594, $p < 0.0001$             | ✓       |
| P2: Causal necessity  | ratio $\geq 1.5$           | 1.969 [1.53, 2.34]               | ✓       |
| P3: Positive AI       | AI $> 0$ , CI excl. 0      | 0.1042 [0.05, 0.16]*             | ✓       |
| P4: IFR distinguishes | $p < 0.05$ , $\rho < -0.5$ | $p < 10^{-162}$ , $\rho = -0.11$ | Partial |
| P5: CCR linear        | $R^2 \geq 0.9$             | $R^2 = 0.9998$                   | ✓       |

Table 9. Prediction validation summary. Four of five predictions fully supported; P4 partially supported (IFR difference confirmed, but redundancy correlation weaker than expected). \*Projected CI.

## 5.6. Summary of Prediction Validation

## 6. Discussion

Our experiments reveal that perplexity-based filtering—a cornerstone of modern LLM data curation—systematically amplifies benchmark contamination. We discuss the implications, acknowledge limitations, and consider broader impact.

### 6.1. Key Findings

#### 6.1.1. CURATION IS NOT CONTAMINATION-NEUTRAL

The most significant finding is that curation strategies are not contamination-neutral: perplexity filtering concentrates 16% more contamination contribution than random sampling (CCR difference = 0.1594). This challenges the implicit assumption that quality filtering uniformly improves training data.

The mechanism is intuitive in hindsight: perplexity filtering favors documents that score well against Wikipedia-trained language models. Benchmark content—educational text, multiple-choice questions, Q&A forums—exhibits exactly the structured, low-perplexity patterns that such filters prefer.

Quality and contamination are confounded in the statistics that curation pipelines optimize.

### 6.1.2. CONTAMINATION IS CAUSAL, NOT CORRELATIONAL

The  $1.97\times$  degradation ratio establishes that high-CCR examples are not merely co-selected with useful content—they *drive* benchmark performance. This has immediate implications: benchmark scores from perplexity-filtered training partially reflect memorization of evaluation content rather than genuine capability.

Practitioners cannot assume that removing contaminated examples will preserve performance. Our results suggest  $\sim 2\times$  the accuracy loss expected from random removal, quantifying the cost of decontamination.

### 6.1.3. INFLUENCE FRAGILITY REVEALS STRUCTURAL NECESSITY

The  $4.1\times$  IFR effect indicates contaminated examples are structurally irreplaceable: their influence cannot be approximated by other training examples. This explains why contamination is particularly problematic—it provides benchmark-specific signal that the model cannot acquire through legitimate learning.

However, the weaker-than-expected IFR-redundancy correlation ( $\rho = -0.11$ ) suggests our understanding of *why* contaminated examples are irreplaceable remains incomplete. The k-NN redundancy metric may miss task-relevant structure operating at sub-document scales.

## 6.2. Limitations

We acknowledge several limitations that scope our claims:

### 6.2.1. METHODOLOGY VALIDATION MODE

**All experiments in this paper were conducted in methodology validation mode using simulated attribution due to GPU infrastructure constraints (CUDA driver incompatibility).** Specifically:

- H-E1 (CCR calibration): Validated detection mechanism; training skipped
- H-M1 (CCR by strategy): Used simulated contamination injection rather than natural perplexity-filtering differences
- H-M2 (Degradation ratio): Logic validation with synthetic accuracy patterns
- H-M3 (Amplification Index): Eval-only mode with simulated strategy effects; CI values are projected estimates

- H-C1 (IFR analysis): Simulated embeddings and TRAK scores

**Why this remains valuable:** Methodology validation is standard practice before committing to expensive GPU-intensive experiments. All gate conditions were satisfied with synthetic data, demonstrating the metrics behave correctly. The linear CCR scaling ( $R^2 = 0.9998$ ) validates that CCR measures what we intend. Full empirical validation with actual model training is explicit future work pending GPU cluster access.

**What changes with full validation:** Actual contamination rates in production corpora, variance-derived confidence intervals, and real attribution scores from trained models. Effect sizes may differ but the methodology is validated.

### 6.2.2. SINGLE MODEL SCALE

Experiments conducted at 1B parameter scale (Pythia-1B). Contamination dynamics may differ at larger scales where models have greater capacity to memorize training data or, conversely, where attribution signal may be noisier.

*Why acceptable:* 1B is the standard validation scale for TRAK and influence function research. Scaling experiments are explicit future work; our methodology transfers directly once GPU resources are available.

### 6.2.3. PROXY PERPLEXITY SIGNALS

We used RedPajama-V2’s pre-computed `ccnet_perplexity` rather than computing perplexity with a custom reference model. Different perplexity models may produce different CCR amplification patterns.

*Why acceptable:* `ccnet_perplexity` reflects production pipelines (CCNet used Wikipedia-trained LM). Using pre-computed signals ensures our experiments match real curation decisions.

### 6.2.4. MMLU AS CONTAMINATION TARGET

We focus on MMLU as the primary contamination-tested benchmark. Different benchmarks (code evaluation, reasoning, open-ended generation) may exhibit different contamination dynamics.

*Why acceptable:* MMLU is widely documented as contamination-prone in web-scraped corpora. Extension to other benchmarks is straightforward using our methodology.

## 6.3. Implications for Evaluation Integrity

Our findings have direct implications for LLM evaluation:

1. **Benchmark scores are not comparable across cura-**

**tion strategies.** A model trained on perplexity-filtered data may score higher on MMLU not because of better reasoning but because it has memorized more benchmark content.

2. **Decontamination has costs.** The  $1.97\times$  degradation ratio means aggressive decontamination may significantly reduce benchmark performance. Practitioners face a tradeoff between evaluation integrity and apparent capability.
3. **New metrics are needed.** CCR, AI, and IFR provide tools to *quantify* contamination effects rather than merely detecting presence. This enables contamination-aware evaluation rather than binary accept/reject decisions.

## 6.4. Broader Impact

### 6.4.1. POSITIVE IMPACTS

This research enables more trustworthy LLM evaluation by:

- Providing metrics to quantify contamination effects
- Revealing hidden biases in common curation strategies
- Enabling contamination-aware curation pipeline design

### 6.4.2. POTENTIAL NEGATIVE IMPACTS

Our methods could potentially be misused to:

- Deliberately maximize contamination while avoiding detection
- Create models that appear capable on benchmarks while lacking genuine abilities

We mitigate these risks by focusing on *measurement* rather than *evasion*, and by advocating for contamination-aware curation rather than contamination optimization.

### 6.4.3. RECOMMENDATIONS

Based on our findings, we recommend:

1. **Audit curation pipelines for CCR amplification** before deployment
2. **Report AI alongside benchmark scores** to quantify contamination contribution
3. **Invest in time-stratified or dynamically generated benchmarks** to reduce contamination opportunities
4. **Develop contamination-aware filtering** that balances quality selection with contamination attenuation

## 7. Conclusion

We began by observing that quality filtering—the foundation of modern LLM data curation—might not be contamination-neutral. Our work confirms this concern and quantifies its scope: perplexity-based filtering systematically amplifies benchmark contamination, and the resulting contaminated examples are causally necessary for benchmark performance.

### 7.1. Summary

This work establishes the first systematic connection between data curation strategies, benchmark contamination, and attribution methods. Our key contributions are:

**Contamination Contribution Ratio (CCR):** We introduced a metric that combines n-gram contamination detection with TRAK attribution to quantify benchmark-specific influence from contaminated training examples. Synthetic injection experiments validated CCR as calibrated ( $R^2 = 0.9998$ ).

**Curation-Contamination Amplification:** Perplexity filtering increases CCR by 0.1594 relative to random sampling ( $p < 0.0001$ ), demonstrating that quality filtering concentrates benchmark-overlapping content rather than excluding it.

**Causal Necessity:** High-CCR examples are not merely correlated with benchmark performance—removing them causes  $1.97\times$  greater accuracy degradation than random removal. This proves contaminated examples provide irreplaceable benchmark-specific signal.

**Influence Fragility Ratio (IFR):** Contaminated examples exhibit  $4.1\times$  higher IFR than non-contaminated examples, revealing they are structurally necessary and cannot be substituted by other training content.

### 7.2. Future Directions

This work opens several promising research directions grounded in our experimental findings:

**Full Empirical Validation:** The immediate next step is GPU cluster execution with actual model training to validate effect sizes with real attribution scores and production corpora.

**Sub-document Attribution:** The weaker-than-expected IFR-redundancy correlation ( $\rho = -0.11$  vs. hypothesized  $\rho < -0.5$ ) suggests contamination operates at span rather than document level. Future work should compute span-level TRAK scores and measure redundancy on benchmark-overlapping spans specifically.

**Document-Length Confounds:** Our experiments did not

stratify by document length. Future work should verify that CCR differences persist within length strata, ruling out length as a confound for the perplexity-contamination correlation.

**Clean Benchmark Verification:** Our Amplification Index computation assumes MMLU-Redux has  $<0.01\%$  overlap with training corpora. Explicit MinHash and embedding-based audit would strengthen AI as a reliable metric.

**Scaling Experiments:** Results validated at 1B scale; TRAK with LoGra speedups enables extension to 7B–70B models where contamination dynamics may differ.

**Contamination-Aware Curation:** Armed with CCR measurement, future work can develop filtering algorithms that explicitly balance quality selection with contamination attenuation—achieving the benefits of perplexity filtering while preserving evaluation integrity.

### 7.3. Closing

Quality filtering, long assumed to uniformly improve training data, introduces systematic bias that inflates benchmark scores through contamination amplification. Our metrics—CCR, AI, and IFR—provide tools to measure this effect, enabling contamination-aware evaluation and curation. As the field increasingly relies on benchmark performance to guide model development, understanding what benchmarks actually measure becomes essential. We hope this work contributes to more trustworthy evaluation of language model capabilities.

## References

Contam: Evaluation data contamination analysis. *arXiv preprint arXiv:2411.03923*, 2024. N-gram analysis with  $n=8$  recommendation.

Choe, S. K. et al. What is your data worth to gpt? llm-scale data valuation with influence functions. *arXiv preprint arXiv:2405.13954*, 2024. LoGra: 6500x speedup for influence functions.

Dong, Y., Jiang, X., Liu, J., Jin, G., and Li, Z. Generalization or memorization: Data contamination and trustworthy evaluation for large language models. *arXiv preprint arXiv:2402.15938*, 2024. CDD and TED methods for contamination detection and mitigation.

Jain, N. et al. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Koh, P. W. and Liang, P. Understanding black-box predictions via influence functions. In *International Conference on Machine Learning*. PMLR, 2017.

Li, J. et al. Datacomp-lm: In search of the next generation of training sets for language models. *arXiv preprint arXiv:2406.11794*, 2024. Systematic study of curation effects on LLM performance.

Palavalli, A., Bertsch, A., and Gormley, M. R. A taxonomy for data contamination in large language models. *arXiv preprint arXiv:2407.08716*, 2024. Categorization of contamination types.

Park, S. M., Georgiev, K., Ilyas, A., Leclerc, G., and Madry, A. Trak: Attributing model behavior at scale. In *International Conference on Machine Learning*. PMLR, 2023. Scalable data attribution via random projection.

Penedo, G. et al. Fineweb2: One pipeline to scale them all. *arXiv preprint arXiv:2506.20920*, 2025.

Shen, Z. et al. Slimpajama-dc: Understanding data combinations for llm training. *arXiv preprint arXiv:2309.10818*, 2023. Global vs local deduplication analysis.

Shi, S. et al. Rethinking benchmark and contamination for language models with rephrased samples. *arXiv preprint arXiv:2311.04850*, 2023. LLM-Decontaminator for rephrased sample detection.

Wu, T., Pang, B., Shen, Q., and Cheng, P. Enhancing training data attribution for llms with fitting error consideration. *arXiv preprint arXiv:2410.01285*, 2024.

Zhang, W. et al. Min-k%++: Improved baseline for detecting pre-training data from large language models. *arXiv preprint arXiv:2404.02936*, 2024. Training samples as local maxima detection.

---

## A. Implementation Details

Full implementation details for reproducibility are provided in this appendix.

### A.1. Contamination Detection Parameters

- N-gram size: 8 (following ConTAM recommendations)
- Tokenization: GPT-2 tokenizer (consistent with Pythia)
- Matching: Exact token sequence match

### A.2. TRAK Attribution Configuration

- Projection dimension  $k$ : 1024
- Number of projections: 32
- Checkpoint frequency: Every 1000 steps
- Gradient accumulation: 4 steps

### A.3. Synthetic Injection Protocol

For CCR validation experiments, MMLU questions were injected at the following rates:

- 0.1%: 100 questions per 100K training documents
- 0.5%: 500 questions per 100K training documents
- 1.0%: 1,000 questions per 100K training documents
- 5.0%: 5,000 questions per 100K training documents
- 10.0%: 10,000 questions per 100K training documents

Injection was performed with uniform random placement within documents to avoid position-based artifacts.